

Random Edge Sampling for Spanner Construction

Mohammadreza Ahmadi

August 28, 2026

Abstract

We study a randomized method to build multiplicative graph spanners using independent edge sampling. For any given graph, we assign a sampling probability to each edge based on its structural properties. Then, we sample the edges independently using these probabilities to create a random subgraph.

Our main goal is to find probability rules that guarantee the resulting subgraph is a spanner of the original graph with high probability. In this paper, we focus on finding a valid probability rule that provides a strict mathematical guarantee. Improving the sparsity of the spanner is our next goal.

1 Introduction

Sparse spanners are subgraphs that keep the shortest-path distances of the original graph almost the same, but with far fewer edges. Peleg and Schäffer [1] formally introduced this concept. A basic problem in spanner theory is to guarantee a good stretch factor while using as few edges as possible.

1.1 Related Work

One very important work in this area is the randomized algorithm by Elkin and Neiman [2]. Their method builds a $(2k - 1)$ -spanner with $O(n^{1+1/k}/\epsilon)$ edges with probability $1 - \epsilon$. It does this using k rounds in the CONGEST model.

1.2 Our Contributions

Our work looks at randomized spanner construction from a different angle. Instead of using randomness in a step-by-step algorithmic process, we ask a simple question: Can we assign the randomness directly to the edges of the input graph?

1.3 Technical Overview

We propose a new framework that assigns a specific sampling probability to every edge. This way, just by sampling the edges independently, we can get a valid spanner with high probability.

2 Preliminaries

Let $G = (V, E)$ be an undirected, unweighted graph with $|V| = n$ and $|E| = m$. For vertices $u, v \in V$, let $\text{dist}_G(u, v)$ denote the shortest-path distance between u and v in G .

Definition 2.1 (Multiplicative Spanner). A subgraph $H = (V, E_H)$, where $E_H \subseteq E$, is a t -spanner of G if, for every pair $u, v \in V$,

$$\text{dist}_G(u, v) \leq \text{dist}_H(u, v) \leq t \cdot \text{dist}_G(u, v).$$

Problem 2.1 (Random Edge-Sampling Spanner Problem). Given a graph $G = (V, E)$ and a target stretch $t \geq 1$, design a probability rule $\mathbf{p} = (p_e)_{e \in E}$ so that the independently sampled subgraph $H_{\mathbf{p}}$ satisfies $\mathbb{P}[H_{\mathbf{p}} \text{ is a } t\text{-spanner of } G] \geq 1 - \delta$, for a small failure probability δ . At the same time, we want to keep the expected number of sampled edges $\mathbb{E}[|E_{\mathbf{p}}|] = \sum_{e \in E} p_e$ as small as possible.

2.1 The Probability Assignment Framework

For every edge $e \in E$, we assign a probability $p_e \in [0, 1]$. We call the set of all these probabilities $\mathbf{p} = (p_e)_{e \in E}$ a *probability assignment*.

We add randomness using independent Bernoulli sampling. For each edge $e \in E$, let $X_e \sim \text{Bernoulli}(p_e)$, where all X_e variables are completely independent from each other. Our final sampled subgraph will be $H_{\mathbf{p}} = (V, E_{\mathbf{p}})$, where $E_{\mathbf{p}} = \{e \in E : X_e = 1\}$.

3 Technical Challenges and Naive Approaches

Before showing our final method, it is helpful to look at some simple ideas that come to mind first. By seeing exactly why these simple ideas fail, we can understand the main challenges of building spanners and see why our final algorithm is necessary.

3.1 The Impossibility of a Global Lower Bound

To make sure that alternative paths survive the random sampling process, the simplest idea is to set a minimum probability for all edges in the graph.

Definition 3.1 (Global Lower Bound Assignment). Given a number $p_{\min} \in [0, 1]$, the global lower bound rule sets the sampling probability of every edge $e \in E$ to be at least $p_{\min}$ ($p_e \geq p_{\min}$).

This idea easily preserves the distances, but it completely fails to make the graph sparse.

Theorem 3.1. Any global lower bound rule that guarantees a t -spanner with probability at least $1 - \delta$ will force the expected size of the spanner to be $\mathbb{E}[|E_{\mathbf{p}}|] = \Omega(m)$.

Proof. Let us look at an edge $e = (u, v)$ that gets dropped during sampling ($X_e = 0$). To keep the stretch factor at most t , an alternative path P_e of length $\ell \leq t$ must completely survive in our new graph $H_{\mathbf{p}}$.

Because we sample edges independently, the chance that the whole path P_e survives is at least $p_{\min}^{\ell}$. So, the chance of failure for this specific edge is at most $1 - p_{\min}^{\ell}$.

To make sure the whole graph is a t -spanner, we cannot let any of the m edges fail. Using the union bound, the total failure probability must be at most δ . This means:

$$m \cdot (1 - p_{\min}^{\ell}) \leq \delta \implies p_{\min}^{\ell} \geq 1 - \frac{\delta}{m}$$

If we take the ℓ -th root and use the generalized binomial theorem for a large m , we get:

$$p_{\min} \geq \left(1 - \frac{\delta}{m}\right)^{1/\ell} \approx 1 - \frac{\delta}{\ell \cdot m}$$

Since $\ell \leq t$ is a constant and δ is very small, the fraction $\frac{\delta}{\ell \cdot m}$ is almost zero. This means $p_{\min}$ must be very close to 1.

The expected number of edges in our new graph is the sum of all probabilities:

$$\mathbb{E}[|E_P|] = \sum_{e \in E} p_e \geq m \cdot p_{\min} \approx m \cdot \left(1 - \frac{\delta}{\ell \cdot m}\right) = \Omega(m)$$

This proves that using a simple global limit gives us a dense graph instead of a sparse one. $\square$

3.2 Deterministic Deletion and Cascading Failures

Theorem 3.1 shows that a global probability does not work, so we must make decisions locally. The easiest local rule is a deterministic one: if an edge $e = (u, v)$ has a short alternative path in the rest of the graph, we just drop it. In math terms, we drop e if the shortest path between u and v in $G \setminus \{e\}$ has a length of at most t .

This idea looks good, but it has a big problem called *cascading failures*. Imagine we drop an edge e_1 because it has an alternative path P_1 . At the same time, another edge e_2 inside P_1 might also be dropped because it has its own alternative path P_2 . If this chain of dropping edges forms a cycle or goes on too long, the whole path P_1 gets destroyed. Because of this, the distance between the endpoints of e_1 becomes much larger than t .

This failure shows that we cannot use simple deterministic rules either. We must use probabilities based on the local structure of the graph.

3.3 The "Good Path" Idea and the Dependency Bottleneck

To solve these problems, let us define a simple concept. For an edge $e = (u, v)$, we say an alternative path between u and v is a *good path* if its length is at most t , and it does not share any edges with other good paths for e (they are edge-disjoint).

Let x_e be the total number of these good paths for the edge e . We can choose a threshold parameter $c = \Theta(\log m)$ and define a simple probability assignment:

Definition 3.2 (Simple Path-Counting Assignment). *If $x_e \leq c$, we keep the edge for sure ($p_e = 1$). If $x_e > c$, we keep it with probability $p_e = c/x_e$.*

This simple idea fixes the density problem because edges with many good paths get a very low probability. However, we quickly hit a major roadblock. Let $S_P = \prod_{e' \in P} p_{e'}$ be the probability that a whole path P survives the sampling process.

The probability that an edge e is dropped AND all its x_e good paths fail is:

$$\mathbb{P}(B_e) = \left(1 - \frac{c}{x_e}\right) \prod_{P \in \mathcal{P}_e} (1 - S_P)$$

Here is the catch: what if a "good path" is made of edges that also have many good paths of their own? If an edge e' in our path has $x_{e'} \rightarrow O(n)$, our rule gives it a probability very close to zero ($p_{e'} \approx 0$).

Because of this, the survival probability of the entire path drops to zero ($S_P \approx 0$). If almost all good paths have $S_P \approx 0$, then $(1 - S_P) \approx 1$. The product term becomes 1, and the failure probability becomes $\mathbb{P}(B_e) \approx 1 - \frac{c}{x_e}$. This is too high and ruins our spanner guarantee.

This bottleneck shows that our first definition of a "good path" was not strong enough. We cannot rely on a path if it uses weak, low-probability edges.

4 The $2c$ -Cut off Framework

To fix the bottleneck, we must upgrade our definition of a good path. An edge should only be dropped if its alternative paths pass through safe and reliable parts of the graph.

4.1 The New Structural Filter

We set a maximum allowed limit for the supporting edges: $x_{\max} = 2c$. Now, for an edge $e = (u, v)$, we define a *safe path* (denoted as $P \in \mathcal{P}_e^*$) if it meets all three conditions:

1. It is an alternative path between u and v that is edge-disjoint from all other safe paths for e .
2. Its length is at most t .
3. Every single edge e' inside this path has $x_{e'} \leq 2c$.

Let $x_e = |\mathcal{P}_e^*|$ be the total number of safe paths for e . Because we added the third rule, we can mathematically guarantee a lower bound on the survival probability of any safe path.

Lemma 4.1 (Safe Path Survival). *For any safe path $P \in \mathcal{P}_e^*$, its survival probability S_P is strictly bounded away from zero, specifically $S_P \geq (1/2)^t$.*

Proof. By the definition of a safe path, every single edge $e' \in P$ is supported by at most $2c$ safe paths in the graph (i.e., $x_{e'} \leq 2c$). Based on our probability assignment rule, the sampling probability of e' is:

$$p_{e'} = \min\left(1, \frac{c}{x_{e'}}\right) \geq \frac{c}{2c} = \frac{1}{2}$$

Since the length of the path P is at most t , it contains at most t edges. The survival probability of the entire independent path is the product of the probabilities of its edges. Therefore:

$$S_P = \prod_{e' \in P} p_{e'} \geq \left(\frac{1}{2}\right)^t$$

This concludes the proof. □

4.2 Bounding the Error and Finding c

Now we can formally bound the probability of failure. The total failure probability is the chance that at least one edge fails to have its distance preserved. We formulate the derivation of our threshold c as the following lemma.

Lemma 4.2 (Failure Probability Bound). *If we set the threshold $c \geq 2^t \ln(m/\delta)$, the sampled graph H_P preserves all pairwise distances up to a factor of t with probability at least $1 - \delta$.*

Proof. Consider a dense edge e ($x_e > c$). The exact probability that the edge is dropped and all its safe paths fail is:

$$\mathbb{P}(B_e) = \left(1 - \frac{c}{x_e}\right) \prod_{P \in \mathcal{P}_e^*} (1 - S_P)$$

To find a safe upper bound, we use the worst-case scenario. Since $x_e > c$, the first term is strictly less than 1 ($(1 - \frac{c}{x_e}) < 1$). Also, because there are x_e paths, multiplying $(1 - S_P)$ by itself x_e times results in a smaller number than multiplying it just c times. Using our guarantee from Lemma 4.1 that $S_P \geq \frac{1}{2^t}$, we can bound the failure probability as:

$$\mathbb{P}(B_e) < 1 \cdot \left(1 - \frac{1}{2^t}\right)^c$$

Using the standard inequality $(1 - x)^y \leq e^{-xy}$, we rewrite this as:

$$\mathbb{P}(B_e) \leq e^{-\frac{c}{2^t}}$$

To ensure the spanner property holds for the whole graph, we must guarantee that *no edge* fails. By applying the union bound over all m edges in the graph, the total probability that the spanner fails is at most:

$$\mathbb{P}(\text{Spanner fails}) \leq \sum_{e \in E} \mathbb{P}(B_e) \leq m \cdot e^{-\frac{c}{2^t}}$$

To guarantee that this total error is at most δ , we require:

$$m \cdot e^{-\frac{c}{2^t}} \leq \delta \implies e^{-\frac{c}{2^t}} \leq \frac{\delta}{m}$$

Taking the natural logarithm of both sides yields:

$$c \geq 2^t \ln \left(\frac{m}{\delta} \right)$$

This concludes the proof. $\square$

4.3 Expected Sparsity Analysis

To ensure the resulting subgraph is sparse while minimizing the overhead, we set our parameter c to be the minimum integer satisfying Lemma 4.2. Thus, we formally define our threshold as $c = \Theta(2^{2k} \log n)$.

Lemma 4.3 (Expected Size Bound). *Using the $2c$ -cutoff strategy with $c = \Theta(2^{2k} \log n)$, the expected number of edges in the sampled spanner is $O(2^{2k} n^{1+1/k} \log n)$.*

Proof. We bound the expected size of the sampled spanner, $\mathbb{E}[|E_p|] = \sum_{e \in E} p_e$, by partitioning the edges of G into two categories: critical edges and dense edges.

1. Critical Edges ($x_e \leq c$): These are the edges that lack a high number of safe alternative paths. For these edges, we set $p_e = 1$. Let $E_{crit} = \{e \in E \mid x_e \leq c\}$ be the set of these edges.

Consider a base case where a subgraph has exactly zero alternative paths of length at most $t = 2k - 1$ for all its edges. In such a subgraph, the shortest cycle containing any edge must have a length of at least $1 + (2k) = 2k + 1$. Thus, the *girth* (the length of the shortest cycle) of this subgraph is strictly greater than $2k$. By the well-known Moore bound [3], any n -vertex graph with a girth greater than $2k$ can have at most $O(n^{1+1/k})$ edges.

In our set E_{crit} , each edge has at most c alternative safe paths of length at most $t = 2k - 1$. We can partition E_{crit} using a greedy decomposition technique [7]. We iteratively extract a maximal subgraph H_i with a girth strictly greater than $2k$, and remove its edges from E_{crit} . Because H_i is maximal, any edge not included in H_i must form a cycle of length at most $2k$ with the edges of H_i . This means every excluded edge has an alternative path of length at most t entirely within H_i .

If we repeat this extraction $c + 1$ times, any remaining edge would possess an alternative path in every single H_i . Since these subgraphs are edge-disjoint, this remaining edge would have at least $c + 1$ edge-disjoint alternative paths, which contradicts the definition of E_{crit} ($x_e \leq c$). Therefore, E_{crit} is completely partitioned into at most $c + 1$ such subgraphs.

Since each subgraph H_i has a girth greater than $2k$, it contains at most $O(n^{1+1/k})$ edges by the Moore bound. The total number of critical edges is thus bounded by the sum of the edges in these $c + 1$ sparse subgraphs:

$$|E_{crit}| \leq (c + 1) \times O(n^{1+1/k}) = O(c \cdot n^{1+1/k})$$

Since $c = \Theta(2^{2k} \log m)$ and $m \leq n^2$, the number of critical edges is $O(2^{2k} n^{1+1/k} \log n)$.

2. Dense Edges ($x_e > c$): For edges with many safe alternative paths, we apply the probability $p_e = \frac{c}{x_e}$. To rigorously bound their expected contribution without assuming uniform

density, we partition the dense edges into logarithmic buckets based on their x_e values. For $j \geq 0$, let $E_j = \{e \in E \mid 2^j c < x_e \leq 2^{j+1} c\}$.

By the exact same logical deduction used for critical edges, if every edge in E_j has at most $2^{j+1} c$ safe paths of length $\leq 2k - 1$, the set E_j can be decomposed into at most $2^{j+1} c + 1$ sparse subgraphs of girth $> 2k$. Thus, by the Moore bound, the absolute maximum number of edges in bucket E_j is tightly bounded by:

$$|E_j| \leq (2^{j+1} c + 1) \times O(n^{1+1/k}) = O(2^j \cdot c \cdot n^{1+1/k})$$

For any edge $e \in E_j$, its sampling probability is $p_e = \frac{c}{x_e} < \frac{c}{2^j c} = \frac{1}{2^j}$. Therefore, the expected number of sampled edges from this specific bucket is:

$$\mathbb{E}[|E_j \cap E_{\mathbf{p}}|] \leq |E_j| \times \frac{1}{2^j} \leq O(2^j \cdot c \cdot n^{1+1/k}) \times \frac{1}{2^j} = O(c \cdot n^{1+1/k})$$

Notice how the exponential growth in the bucket size $|E_j|$ is perfectly canceled out by the exponential decay in the sampling probability $\frac{1}{2^j}$. Since the maximum possible value for x_e is bounded by $m \leq n^2$, there are at most $O(\log n)$ such buckets. Summing the expected edges across all $\log n$ buckets yields:

$$\sum_{\text{dense } e} p_e \leq \sum_{j=0}^{\log n} O(c \cdot n^{1+1/k}) = O(c \cdot n^{1+1/k} \log n)$$

Substituting $c = \Theta(2^k \log n)$ gives $O(2^k n^{1+1/k} \log^2 n)$. By adding the expected bounds for both critical and dense edges together, the total expected number of edges in our random spanner is asymptotically $O(2^k n^{1+1/k} \log^2 n)$. $\square$

5 Regime of Efficacy and Structural Limitations

Our randomized framework establishes a direct trade-off between the stretch factor and the sparsity threshold. However, relying purely on independent edge sampling introduces inherent theoretical boundaries.

5.1 Regime of Efficacy and Optimal Sparsity

The presence of the 2^{2k} factor in our expected size bound highlights a fundamental limitation of purely independent edge sampling. Because edges are sampled independently, the probability of a whole path surviving decays exponentially with its length $t = 2k - 1$. To mathematically guarantee that at least one alternative path survives without using dependent clustering or global BFS trees, we are forced to overcompensate by setting an exponentially large path-counting threshold $c = \Theta(2^{2k} \log n)$. This structural overcompensation inevitably inflates the expected size of the spanner for large stretch factors.

Despite this asymptotic exponential dependence on k , the framework remains highly efficacious in practical domains. For small constant stretch parameters prevalent in real-world network routing, such as $k \in \{2, 3, 4\}$, the term 2^{2k} acts as a small constant (16, 64, and 256 respectively). In these restricted regimes, our framework independently samples ultra-sparse spanners of size $O(n^{1.5} \log^2 n)$, $O(n^{1.33} \log^2 n)$, and $O(n^{1.25} \log^2 n)$ without requiring complex dependent sampling or sequential clustering phases.

Furthermore, the global optimal behavior of this framework can be determined by minimizing the core sparsity term $E(k) = 2^{2k} n^{1/k}$ with respect to k . Taking the base-2 logarithm yields

$\log_2 E(k) = 2k + \frac{\log_2 n}{k}$. Differentiating with respect to k and setting the derivative to zero yields the optimal scaling parameter:

$$k^* = \sqrt{\frac{\log_2 n}{2}} = \Theta(\sqrt{\log n})$$

Substituting this optimal k^* back into the expected size bound yields a spanner size of $O(n \cdot 2^{2\sqrt{2\log_2 n}} \log^2 n)$, which simplifies asymptotically to $n^{1+o(1)}$. This proves that our independent sampling framework achieves its maximum efficiency at $k = \Theta(\sqrt{\log n})$, producing nearly linear-sized subgraphs, and maintains valid sub-quadratic spanners for any $k < \frac{1}{2} \log_2 n$.

6 Polynomial-Time Algorithmic Realization

As we established, our independent sampling framework uses the exact parameter x_e (the maximum number of edge-disjoint safe paths) to guarantee a sparse spanner. However, exactly computing the maximum number of edge-disjoint paths of bounded length is well-known to be NP-hard for $t \geq 4$ [6].

In this section, we show how to bypass this hardness using a two-phase greedy approximation. This approach runs in polynomial time and acts as a valid surrogate for x_e without breaking any of our probabilistic guarantees.

6.1 The Two-Phase Greedy Algorithm

The main difficulty in finding safe paths is a cyclic dependency: to know if a path is safe, we must know if its edges are safe ($x_{e'} \leq 2c$), which in turn requires knowing the path densities of those edges. We break this cycle using a simple two-pass strategy.

Phase 1: Finding an Upper Bound. For each edge $e = (u, v) \in E$, we run a standard Breadth-First Search (BFS) on $G \setminus \{e\}$ to repeatedly extract and remove paths of length at most t . We stop when no such paths remain. Let $q_e^{(0)}$ be the total number of extracted paths.

Since this greedy approach finds a maximal set of valid edge-disjoint paths, $q_e^{(0)}$ is clearly an upper bound on the true number of *safe* paths. Thus, if $q_e^{(0)} \leq 2c$, we are certain that e is a safe edge. If $q_e^{(0)} > 2c$, we cannot be sure, so we temporarily mark the edge e as heavy.

Phase 2: Extracting the Actual Safe Paths. Now we want to count paths that only use verified safe edges. We temporarily delete all the heavy edges identified in Phase 1 from the graph. Then, for every edge $e \in E$, we repeat the same iterative BFS extraction on the remaining graph.

We define q_e as the final number of extracted paths in this phase. Since the graph now only contains definitively safe edges, any path found here is guaranteed to be a safe path.

6.2 Approximation Guarantee and Complexity

Lemma 6.1 (Greedy t -Approximation). *Let x_e be the optimal maximum number of edge-disjoint safe paths of length at most t . The surrogate value q_e computed in Phase 2 satisfies $\frac{x_e}{t} \leq q_e \leq x_e$.*

Proof. Let OPT be the optimal maximum set of edge-disjoint safe paths, with $|OPT| = x_e$. Let $GREEDY$ be the set of paths found in Phase 2, with $|GREEDY| = q_e$. Since Phase 2 only explores the verified safe edges and builds valid edge-disjoint paths, it is a feasible solution to the packing problem. Thus, $q_e \leq x_e$.

For the lower bound, notice that our iterative BFS only stops when no more valid paths exist, making the set $GREEDY$ a maximal set. Because it is maximal, every path $P^* \in OPT$ must share at least one edge with some path $P \in GREEDY$.

The length of any path $P \in \text{GREEDY}$ is bounded by t , so it contains at most t edges. Since all paths in OPT are mutually edge-disjoint, a single edge can belong to at most one path in OPT . This means one path $P \in \text{GREEDY}$ can intersect (and therefore block) at most t distinct paths in OPT . Since every path in OPT is blocked by the GREEDY set, the total number of optimal paths is at most $t \cdot q_e$, which gives $q_e \geq \frac{x_e}{t}$. $\square$

Time Complexity Analysis: For a given edge $e = (u, v)$, identifying a single path of length at most t requires a standard Breadth-First Search (BFS) truncated at depth t . A single BFS traversal takes $O(m)$ time. Upon finding a valid path, we delete its edges from the temporary graph and repeat the BFS. Since the graph contains m edges and each successful iteration removes at least one edge, the BFS is invoked at most $O(m)$ times per edge. Thus, the iterative extraction takes $O(m) \times O(m) = O(m^2)$ time per edge. Executing this two-phase process over all m edges yields a total polynomial time complexity of $O(m^3)$.

6.3 Preserving the Framework Bounds

To make the framework operational, we replace the exact parameter x_e with our polynomial-time surrogate q_e . The applied sampling probability for each edge becomes:

$$\hat{p}_e = \min \left(1, \frac{c}{q_e} \right)$$

We must verify that this substitution strictly satisfies the mathematical proofs established in our framework:

- **Lemma 4.1 (Safe Path Survival):** A safe path consists of edges where the bounded density is at most $2c$. Our Phase 1 explicitly identifies and isolates safe edges by ensuring their raw upper bound satisfies $q_{e'}^{(0)} \leq 2c$. Because Phase 2 operates on a restricted subgraph, the refined count naturally satisfies $q_{e'} \leq q_{e'}^{(0)} \leq 2c$. Consequently, the new probability applied to these safe edges is $\hat{p}_{e'} = \min \left(1, \frac{c}{q_{e'}} \right) \geq \min \left(1, \frac{c}{2c} \right) = \frac{1}{2}$. Thus, the strict lower bound $S_P \geq (1/2)^t$ holds perfectly.
- **Lemma 4.2 (Failure Probability Bound):** For dense edges, the failure probability fundamentally relies on the term $(1 - \frac{c}{x_e})$. By Lemma 6.1, we proved $q_e \leq x_e$. Therefore, $\frac{c}{q_e} \geq \frac{c}{x_e}$, which mathematically implies $(1 - \frac{c}{q_e}) \leq (1 - \frac{c}{x_e})$. This means the applied substitution actually decreases the failure probability, making our union bound of $m \cdot e^{-\frac{c}{2t}} \leq \delta$ even more secure. The stretch is strictly guaranteed.
- **Lemma 4.3 (Expected Sparsity):** The expected number of edges depends on the sum of sampling probabilities. From the lower bound of Lemma 6.1, we know $q_e \geq \frac{x_e}{t}$. Consequently, the inflated probability is bounded by:

$$\hat{p}_e = \min \left(1, \frac{c}{q_e} \right) \leq \min \left(1, \frac{t \cdot c}{x_e} \right) \leq t \cdot \min \left(1, \frac{c}{x_e} \right) = t \cdot p_e$$

The expected number of sampled edges is scaled by at most $t = 2k - 1$. The theoretical bound becomes $O(k \cdot 2^{2k} n^{1+1/k} \log n)$, seamlessly maintaining the asymptotic sub-quadratic sparsity in terms of n .

7 Conclusion and Future Work

In this work, we introduced a randomized framework for constructing multiplicative graph spanners using independent edge sampling. By defining safe paths and using a $2c$ -cutoff strategy, we removed the survival dependencies between edges. We showed that assigning sampling probabilities based on x_e guarantees a $(2k - 1)$ -spanner with an expected size of $O(2^{2k} n^{1+1/k} \log n)$. Furthermore, we identified the optimal scaling regime at $k = \Theta(\sqrt{\log n})$, where the framework produces near-linear $n^{1+o(1)}$ size spanners. We also showed how to implement this framework in polynomial time using a simple two-phase greedy approach that preserves all mathematical guarantees.

This framework leaves a few interesting directions for future work:

- **Tightening the Approximation Ratio:** While our greedy algorithm gives a t -approximation to bypass the NP-hard bottleneck, it is worth investigating if other relaxations (like fractional flow rounding) can provide a tighter approximation ratio $\alpha < t$ in polynomial time, which would improve the constant factors in the sparsity bound.
- **Optimizing the Cutoff Parameter:** Our structural filter uses a maximum limit of $2c$ to bound the survival probability away from zero ($S_p \geq 2^{-t}$). Generalizing this threshold to αc (for $\alpha > 1$) creates a trade-off: a higher cutoff increases the survival probability of paths but also increases the expected sparsity. Finding the optimal value for this cutoff is an open question.
- **Tightening the Sparsity Bound:** Our randomized sampling framework currently yields an expected sparsity containing an exponential 2^{2k} factor, which limits its applicability for large k . A natural theoretical question is whether this factor can be entirely removed to match the classic extremal bound of $O(n^{1+1/k})$. It remains to be seen if this exponential penalty is a strict inherent requirement of the independent probability assignment or an artifact of the union bound that can be bypassed.
- **Beyond Independent Sampling:** We restricted our model to independent Bernoulli sampling to keep the analysis simple and avoid cascading failures. However, using dependent sampling schemes or randomized rounding techniques to reduce the number of edges without breaking the spanner guarantee is an exciting direction for future research.

References

- [1] D. Peleg and A. A. Schäffer, “Graph spanners,” *Journal of Graph Theory*, vol. 13, no. 1, pp. 99–116, 1989.
- [2] M. Elkin and O. Neiman, “Efficient algorithms for constructing very sparse spanners and emulators,” *ACM Transactions on Algorithms (TALG)*, vol. 15, no. 1, pp. 1–29, 2019.
- [3] N. Alon, S. Hoory, and N. Linial, “The Moore bound for irregular graphs,” *Graphs and Combinatorics*, vol. 18, no. 1, pp. 53–57, 2002.
- [4] M. Mitzenmacher and E. Upfal, *Probability and Computing: Randomization and Probabilistic Techniques in Algorithms and Data Analysis*. Cambridge University Press, 2017.
- [5] K. Menger, “Zur allgemeinen Kurventheorie,” *Fundamenta Mathematicae*, vol. 10, no. 1, pp. 96–115, 1927.

- [6] A. Itai, Y. Perl, and Y. Shiloach, "The complexity of finding maximum disjoint paths with length constraints," *Networks*, vol. 12, no. 3, pp. 277–286, 1982.
- [7] I. Althöfer, G. Das, D. Dobkin, D. Joseph, and J. Soares, "On sparse spanners of weighted graphs," *Discrete & Computational Geometry*, vol. 9, no. 1, pp. 81–100, 1993.
- [8] G. L. Miller, R. Peng, I. Vladu, and S. Xu, "Improved parallel algorithms for spanners and hopsets," *Proceedings of the 27th ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*, pp. 192–201, 2015.